

map()

Transform every item in a sequence

Python built-in · one function, applied to all

```
[1, 2, 3]
  |
  x * 2
  |
[2, 4, 6]
```

What is map()?

It applies a function to every item, so you don't write the loop

```
map(function, iterable)
```

1

function

what to do with ONE item
(a lambda, a built-in, or your own def)

2

iterable

the list / tuple / string
to run through

3

returns

a lazy map object
- wrap it in list()

The idea

A loop says HOW to walk the list.

map() says WHAT to do with each item, and Python does the walking.

Same result, less plumbing.

Example 1: double every number

The same job written three ways

for loop

```
nums = [1, 2, 3, 4, 5]
doubled = []

for n in nums:
    doubled.append(n * 2)
```

5 lines

map()

```
nums = [1, 2, 3, 4, 5]

doubled = list(
    map(lambda x: x * 2,
        nums))
```

1 expression

comprehension

```
nums = [1, 2, 3, 4, 5]

doubled = [
    x * 2
    for x in nums]
```

most Pythonic

All three print

[2, 4, 6, 8, 10]

Example 2: map with built-in functions

No lambda needed - just pass the function's name

str.upper

```
names = ["subha", "roberto", "ayse"]  
list(map(str.upper, names))
```

-> ['SUBHA', 'ROBERTO', 'AYSE']

len

```
names = ["subha", "roberto", "ayse"]  
list(map(len, names))
```

-> [5, 7, 4]

float

```
prices = ["2.50", "3.75", "1.20"]  
list(map(float, prices))
```

-> [2.5, 3.75, 1.2]

your own function

```
def tax(p):  
    return round(p * 1.19, 2)  
list(map(tax, [10, 20]))
```

-> [11.9, 23.8]

map() on our own class code

The exercises we already solved with loops, now in one line

alphabetor (day 1 exercise)

```
names = ["Subha", "Chakra",  
         "Amlaki", "biplob"]  
list(map(alphabetor, names))
```

-> [-1, -1, 1, 0]

square (lambda practice)

```
square = lambda x: x ** 2  
  
list(map(square, [1, 2, 3, 4, 5]))
```

-> [1, 4, 9, 16, 25]

digitize (codewars kata)

```
list(map(int, str(8675309)))  
  
# each character -> its own digit
```

-> [8, 6, 7, 5, 3, 0, 9]

invert (codewars kata)

```
list(map(lambda n: -n,  
         [1, 2, 3, 4, 5]))
```

-> [-1, -2, -3, -4, -5]

Same answers as our loops - map() just moves the walking into one line.

Example 3: two lists at once

map() can walk several sequences in parallel

```
a = [1, 2, 3]
b = [10, 20, 30]

list(map(lambda x, y: x + y, a, b))
-> [11, 22, 33]
```

Pairs up item by item

```
1 + 10  -> 11
2 + 20  -> 22
3 + 30  -> 33
```

Watch out: uneven lengths

```
list(map(lambda x, y: x + y, [1, 2, 3], [10, 20])) -> [11, 22]
```

map stops at the shortest list - the leftover 3 is simply dropped.

The one gotcha: map is lazy

A map object gives up its items once, then it's empty

```
m = map(lambda x: x * 2, [1, 2, 3])

print(m)
-> <map object at 0x...>

print(list(m))
-> [2, 4, 6]

print(list(m))
-> []          # already used up
```

Remember

Nothing is computed until you ask for the items.

Wrap it in `list()` once, and keep that list.

Lazy = memory friendly: a map over a million rows computes one item at a time.

map vs filter vs reduce

Three tools, three different jobs

map()

changes every item

```
list(map(
    lambda x: x * 2,
    [1, 2, 3, 4, 5]))
```

-> [2, 4, 6, 8, 10]

same length out

filter()

keeps some items

```
list(filter(
    lambda x: x % 2 == 0,
    [1, 2, 3, 4, 5]))
```

-> [2, 4]

shorter or equal

reduce()

boils down to one

```
reduce(
    lambda x, y: x + y,
    [1, 2, 3, 4, 5])
```

-> 15

a single value

reduce needs an import: `from functools import reduce`

Takeaways

- 1 `map(function, iterable)` applies one function to every item
- 2 Pass a lambda, a built-in like `str.upper`, or your own function by name
- 3 Wrap the result in `list()` - a map object is lazy and one-shot
- 4 Several iterables? `map` pairs them up and stops at the shortest
- 5 For simple cases a list comprehension is often more readable

Next up in Group 5: `filter()` and `reduce()`